

October 26, 2020

## 1 Fonaments de Programació

### 1.1 Unitat 4: Llibreries científiques en Python - Exercicis resultats

En aquest Notebook hi trobareu un conjunt d'exercicis per practicar. Aquests exercicis no puntuen per a la PAC, però us recomanem que els intenteu resoldre com a part del procés d'aprenentatge. Trobareu exemples de possibles solucions als exercicis al propi notebook, però és important que intenteu resoldre'ls vosaltres abans de consultar les solucions. Les solucions us han de permetre validar les vostres respostes, així com veure alternatives de resolució de les activitats. També us animem a preguntar qualsevol dubte que us sorgeixi sobre la resolució de les activitats per practicar al fòrum de l'aula.

---

### 1.2 Preguntes i exercicis per a practicar

#### 1.2.1 Exercici 1

Calculeu la norma i el determinant de la següent matriu:  $\begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix}$ .

```
[2]: # Resposta
```

#### 1.2.2 Exercici 2

Genereu una llista de 100 valors enters aleatoris de 0-9. Realitzeu els següents càlculs utilitzant mètodes de *numpy* : - Mitjana i desviació estàndard dels valors de la llista - Valor màxim i mínim - Sumeu tots els valors de la llista - Aconseguiu una llista de valors únics

```
[3]: # Resposta
```

#### 1.2.3 Exercici 3

Creeu una matriu 10x10 que correspongui amb la [matriu identitat](#) utilitzant generadors bàsics d'arrays. Creeu una matriu identitat 10x10 (aquest cop utilitzant generadors específics de matrius identitat) i comproveu que ambdues matrius són iguals.

**Consideracions:** - La primera matriu ha de crear-se utilitzant constructors bàsics d'arrays, com els presentats als Notebooks de teoria. \* La segona matriu ha de generar-se utilitzant el generador de matrius identitat de numpy. \* La comparació ha de retornar True si les matrius són iguals (un únic True), False en cas contrari.

[4]: `# Resposta`

#### 1.2.4 Exercici 4

Ordeneu la matriu bidimensional `[[5,1,7], [0,7,4], [7,23,1]]` per columnes utilitzant com a algorisme d'ordenació el [Heapsort](#).

**Nota:** Per resoldre aquest exercici us pot ser d'utilitat la funció [sort](#) de Numpy que ens permet ordenar els elements d'un array N-dimensional.

[5]: `# Resposta`

#### 1.2.5 Exercici 5

Creeu una matriu, I, de mida 10x10 on tots els seus elements tindran el valor 0 utilitzant generadors de matrius de numpy. Modifiqueu la matriu anterior de tal forma que `I[i, j] = i * j` per tot i i j parells, és a dir, per a totes les posicions on tant i com j són parells.

[6]: `# Resposta`

#### 1.2.6 Exercici 6

Considereu la matriu `[[1, 2, 3], [4, 5, 6]]`. Transformeu-la en la matriu 1D `[1, 2, 3, 4, 5, 6]`.

**Nota:** Per resoldre aquest exercici us poden ser d'utilitat les funcions [ravel](#) o [flatten](#) de Numpy que ens permeten transformar un array 2D numpy a 1D.

[7]: `# Resposta`

#### 1.2.7 Exercici 7

Creeu una matriu, Z, de mida 8x8 que vagi alternant els valors 0 i 1, seguint un patró de taulell de dames.

[8]: `# Resposta`

### 1.2.8 Exercici 8

L'arxiu `us_births.csv` conté dades dels naixements a Estats Units durant els anys 1994-2003. Carregueu l'arxiu representeu de forma gràfica els naixements agrupats per mes i any. Utilitzeu un gràfic 2D de tipus *scatter*.

*Consideracions:* \* La figura ha de contenir 10 gràfics (1 per any) \* El valor de cada punt ha de correspondre amb el total de naixements per any i mes. \* Incloeu una llegenda que contingui els anys. Podeu diferenciar cada gràfic como millor us sembli (diferents símbols / colors / combinació de les dues)

[9]: `# Resposta`

### 1.2.9 Exercici 9

Carregueu les dades de l'arxiu `company_sales.csv`, que podeu trobar a la carpeta `data`, en forma de dataframe. Aquest conjunt de dades recull informació sobre les vendes mensuals d'una empresa de cosmètica i productes d'higiene.

Part A:

Mostreu un gràfic de línies de punts de les vendes totals (*total\_profit*) per cada mes (els mesos estan numerats d'1 a 12).

*Consideracions:*

- La línia ha de ser amb punts (*dotted line*) de color vermell i de grandària 3.
- S'ha de mostrar la llegenda a la part inferior dreta.

Part B:

Mostreu un gràfic circular (*pie chart*) amb el número d'unitats venudes aquest any per producte en percentatge.

*Consideracions:*

- Heu d'incloure el percentatge (%) de cada producte dins del seu respectiu espai del gràfic.
- S'ha de mostrar la llegenda a la part dreta.

[10]: `# Resposta`

### 1.2.10 Exercici 10

Carregueu les dades de l'arxiu `got.csv` a un *dataframe*. Aquest conjunt de dades recull informació de la [Guerra dels Cinc Reis](#) de les novel·les de [Cançó de gel i de foc](#) de George R.R. Martin.

Mostreu el número de files del *dataframe* i les etiquetes dels eixos.

[11]: `# Resposta`

### 1.2.11 Exercici 11

Agrupeu les dades carregades a l'exercici 1 pel principal bàndol atacant (*attacker\_1*). Per cada posició, mostreu el número de batalles i el resultat de la batalla (el resultat es troba al camp *attacker\_outcome*).

```
[12]: # Resposta
```

### 1.2.12 Exercici 12

Mostreu les dades de las batalles on el número de participants supera els 15000 “homes” (comptant els dos bàndols: *attacker\_size* i *defender\_size*), el resultat hagi estat desfavorable per l'atacant, i la batalla hagi estat a l'hivern ( *summer* = 0).

```
[13]: # Resposta
```

### 1.2.13 Exercici 13

Compteu quants llocs apareixen més d'un cop al *dataframe* (camp *location*), utilitzant les funcions de la llibreria *pandas*. Existeix algun lloc on hi hagi hagut més d'una batalla? Comproveu quins bàndols hi estaven implicats.

```
[14]: # Resposta
```

### 1.2.14 Exercici 14

Carregueu les dades de l'arxiu *netflixtitles.csv*, que podeu trobar a la carpeta **data**, en un *dataframe*. Aquest conjunt de dades recull informació sobre pel·lícules i sèries de TV de [Netflix](#) fins el 2019.

Mostreu les pel·lícules estrenades el 2019 que tenen una duració superior a 100 minuts. Heu de mostrar com a resultat una llista dels títols **sense repetir**.

**Nota:** Al Notebook de teoria hem vist operacions bàsiques de filtrat. Per resoldre aquest exercici us caldrà investigar com aplicar condicions més complexes.

**Nota 2:** Els valors de la columna de duració (*duration*) són strings, ja que combinen números i lletres. Per tant, s'han de transformar a int un cop s'hagi construït el subconjunt de dades de pel·lícules. Us pot ser d'utilitat la funció *astype* de *pandas*.

```
[15]: # Resposta
```

## 1.3 Solucions als exercicis per a practicar

### 1.3.1 Exercici 1

Calculeu la norma i el determinant de la següent matriu:  $\begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix}$ .

```
[16]: # Carreguem les llibreries necessàries
import numpy as np

# Creem la matriu com a array bidimensional
m = np.array([[1, 0], [2, -1]])

# Calculem la norma i el determinant utilitzant el mòdul linalg
norm = np.linalg.norm(m)
det = np.linalg.det(m)

print("La matriu m és:", m)
print("La norma de m és: ", norm)
print("El determinant de m és:", det)
```

La matriu m és:  $\begin{bmatrix} 1 & 0 \\ 2 & -1 \end{bmatrix}$

La norma de m és: 2.449489742783178

El determinant de m és: -1.0

### 1.3.2 Exercici 2

Genereu una llista de 100 valors enters aleatoris de 0-9. Realitzeu els següents càlculs utilitzant mètodes de *numpy* : - Mitjana i desviació estàndard dels valors de la llista - Valor màxim i mínim - Sumeu tots els valors de la llista - Aconseguiu una llista de valors únics

```
[17]: # Carreguem les llibreries necessàries
from random import randint
import numpy as np

# Utilitzem la funció randint del mòdul random i list comprehensions per a
↳ generar la llista, com ja havíem vist a
# unitats anteriors
rand_ints = [randint(0, 9) for _ in range(100)]

# Per a calcular la mitjana i la desviació estàndard utilitzem les funcions
↳ mean y std (standard deviation)
# respectivament
m = np.mean(rand_ints)
std = np.std(rand_ints)

print("La nostra llista de valors aleatoris és:", rand_ints)
```

```

print("La seva mitjana és %f, i la seva desviació estàndard és %f" %(m, std))

# Per als valors màxim i mínim, utilitzem max i min
max_val = np.max(rand_ints)
min_val = np.min(rand_ints)

print("Els seus valors màxim i mínim són, respectivament: %d i %d" %(max_val,
    ↳min_val))

# Per a sumar tots els elements de la llista, la funció sum
sum_values = np.sum(rand_ints)

print("El resultat de la suma de tots els valors de la llista és", sum_values)

# I finalment per a aconseguir una llista de valors únics, la funció unique
unique_values = np.unique(rand_ints)

print("I la llista de valors únics és", unique_values)

```

La nostra llista de valors aleatoris és: [7, 8, 8, 4, 7, 5, 9, 9, 0, 4, 0, 2, 3, 5, 8, 0, 0, 1, 5, 9, 1, 6, 1, 2, 9, 4, 2, 8, 1, 4, 0, 8, 1, 9, 7, 9, 9, 0, 7, 3, 9, 0, 9, 2, 4, 7, 9, 9, 9, 5, 2, 9, 9, 2, 4, 5, 9, 2, 2, 6, 7, 0, 6, 0, 9, 2, 8, 2, 9, 9, 6, 0, 1, 1, 2, 7, 3, 1, 9, 5, 9, 2, 4, 5, 5, 8, 7, 4, 1, 9, 9, 1, 8, 1, 7, 0, 0, 7, 4, 9]

La seva mitjana és 4.850000, i la seva desviació estàndard és 3.296589

Els seus valors màxim i mínim són, respectivament: 9 i 0

El resultat de la suma de tots els valors de la llista és 485

I la llista de valors únics és [0 1 2 3 4 5 6 7 8 9]

### 1.3.3 Exercici 3

Creeu una matriu 10x10 que correspongui amb la [matriu identitat](#) utilitzant generadors bàsics d'arrays. Creeu una matriu identitat 10x10 (aquest cop utilitzant generadors específics de matrius identitat) i comproveu que ambdues matrius són iguals.

**Consideracions:** - La primera matriu ha de crear-se utilitzant constructors bàsics d'arrays, com els presentats als Notebooks de teoria. \* La segona matriu ha de generar-se utilitzant el generador de matrius identitat de numpy. \* La comparació ha de retornar True si les matrius són iguals (un únic True), False en cas contrari.

```

[18]: # Carreguem les llibreries necessàries
import numpy as np

# Podem definir la matriu de forma manual com una llista de llistes
m0 = np.array([[1, 0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
               [0, 0, 1, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],

```

```

        [0, 0, 0, 0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]])

# O començant per una 10x10 amb tot zeros i assignar uns a la diagonal
# recorrent la matriu per files i columnes
m1 = np.zeros((10,10), dtype=int)
for i, r in enumerate(m1):
    r[i] = 1

# Per altra banda, definim la identitat 10x10 utilitzant el constructor
↳ "built-in"
m2 = np.identity(10, dtype=int)

# I realitzem la comparació. Per fer-ho podem utilitzar el mètode array_equal
↳ de numpy
# que ens compara si dos arrays són iguals
print(np.array_equal(m0, m2))
print(np.array_equal(m1, m2))

# També podem comprovar que la comparació falla si modifiquem qualsevol valor
↳ de la matriu
m0[1, 2] = 1
print(np.array_equal(m0, m2))

```

True  
True  
False

### 1.3.4 Exercici 4

Ordeneu la matriu bidimensional `[[5,1,7], [0,7,4], [7,23,1]]` per columnes utilitzant com a algorisme d'ordenació el [Heapsort](#).

**Nota:** Per resoldre aquest exercici us pot ser d'utilitat la funció `sort` de Numpy que ens permet ordenar els elements d'un array N-dimensional.

```

[19]: # Carreguem les llibreries necessàries
import numpy as np

# Creem la matriu indicada a l'enunciat
matriu_ex3 = [[5,1,7], [0,7,4], [7,23,1]]

# Utilitzem la funció sort de numpy per ordenar la matriu anterior.
# El paràmetre axis = 0 ens indica que volem ordenar per columnes
# i el paràmetre kind ens permet especificar l'algorisme d'ordenació, en aquest
↳ cas 'heapsort'.
np.sort(matriu_ex3, axis=0, kind='heapsort')

```

```
[19]: array([[ 0,  1,  1],
           [ 5,  7,  4],
           [ 7, 23,  7]])
```

### 1.3.5 Exercici 5

Creeu una matriu, I, de mida 10x10 on tots els seus elements tindran el valor 0 utilitzant generadors de matrius de numpy. Modifiqueu la matriu anterior de tal forma que  $I[i, j] = i * j$  per tot i i j parells, és a dir, per a totes les posicions on tant i com j són parells.

```
[20]: # Carreguem les llibreries necessàries
import numpy as np

# Definim un parell de constants pel nombre de files i columnes
NUM_FILES = 10
NUM_COLUMNES = 10

# Creem una matriu de 10x10 on tots els valors seran 0.
# Per aconseguir-ho utilitzem la funció zeros que hem vist al Notebook de
↳ teoria.
matriu_ex4 = np.zeros((NUM_FILES, NUM_COLUMNES))

# Mostrem la matriu original per pantalla
print(matriu_ex4)

# Iterem sobre totes les files i columnes de la matriu generada utilitzant un
↳ doble bucle for.
# El primer bucle ens recorrerà l'index de les files, mentre que el segon bucle
↳ ens recorrerà
# l'index de les columnes.
for i in range(0, NUM_FILES):
    for j in range(0, NUM_COLUMNES):
        # Comprovem que l'index de la fila i l'index de la columna siguin tots
↳ dos parells.
        if i % 2 == 0 and j % 2 == 0:
            # En cas afirmatiu, passem a realitzar l'assignació indicada.
            matriu_ex4[i, j] = i * j

# Mostrem el resultat final per pantalla
print(matriu_ex4)
```

```
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```



```

[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  4.  0.  8.  0. 12.  0. 16.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  8.  0. 16.  0. 24.  0. 32.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. 12.  0. 24.  0. 36.  0. 48.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. 16.  0. 32.  0. 48.  0. 64.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]]

```

### 1.3.6 Exercici 6

Considereu la matriu  $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ . Transformeu-la en la matriu 1D  $[1, 2, 3, 4, 5, 6]$ .

**Nota:** Per resoldre aquest exercici us poden ser d'utilitat les funcions `ravel` o `flatten` de Numpy que ens permeten transformar un array 2D numpy a 1D.

```

[21]: # Carreguem les llibreries necessàries
import numpy as np

# Creem la matriu indicada a l'enunciat
matriu_ex6 = np.array([[1, 2, 3], [4, 5, 6]])

#Opció 1: ravel
out1 = np.ravel(matriu_ex6)
print('Opció 1: ravel\n',out1)

#Opció 2: flatten
out2 = matriu_ex6.flatten()
print('Opció 2: flatten\n',out1)

```

```

Opció 1: ravel
[1 2 3 4 5 6]
Opció 2: flatten
[1 2 3 4 5 6]

```

### 1.3.7 Exercici 7

Creeu una matriu, Z, de mida 8x8 que vagi alternant els valors 0 i 1, seguint un patró de taulell de dames.

```
[22]: # Carreguem les llibreries necessàries
import numpy as np
import matplotlib.pyplot as plt

# Creem una matriu de zeros 8x8
Z = np.zeros((8,8),dtype=int)

# Començant en la fila 1 i cada 2, posem 1 cada 2 columnes
Z[1::2,::2] = 1

# Començant en columna 1 i cada 2, posem 1 cada 2 files
Z[:,1::2] = 1
print(Z)
```

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

### 1.3.8 Exercici 8

L'arxiu `us_births.csv` conté dades dels naixements a Estats Units durant els anys 1994-2003. Carregueu l'arxiu representeu de forma gràfica els naixements agrupats per mes i any. Utilitzeu un gràfic 2D de tipus *scatter*.

*Consideracions:* \* La figura ha de contenir 10 gràfics (1 per any) \* El valor de cada punt ha de correspondre amb el total de naixements per any i mes. \* Incloeu una llegenda que contingui els anys. Podeu diferenciar cada gràfic como millor us sembli (diferents símbols / colors / combinació de les dues)

```
[23]: # Carreguem les llibreries necessàries
%matplotlib inline
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Carreguem les dades del fitxer
data = pd.read_csv('data/us_births.csv')

# Les agrupem per any i mes
grouped = data.groupby(['year', 'month'])
```

```

# Grouped conté un conjunt de dataframes identificats per any i mes, però les
↳ dades
# referents al naixements a cada dataframe encara estan separades per dia.
# Sumem els naixements de cada mes.
parsed_years = grouped['births'].sum()

# Separem les dades en una matriu de 10x12 (10 anys amb 12 mesos per any)
parsed_years = np.array(parsed_years).reshape(10, 12)

# Arribats a aquest punt ja tenim les dades llistes per ser mostrades, ara ens
↳ toca
# definir els atributs del gràfic (eixos, colors, etc.)

# Representarem els mesos con una llista de 1 a 12
months = range(1, 13, 1)

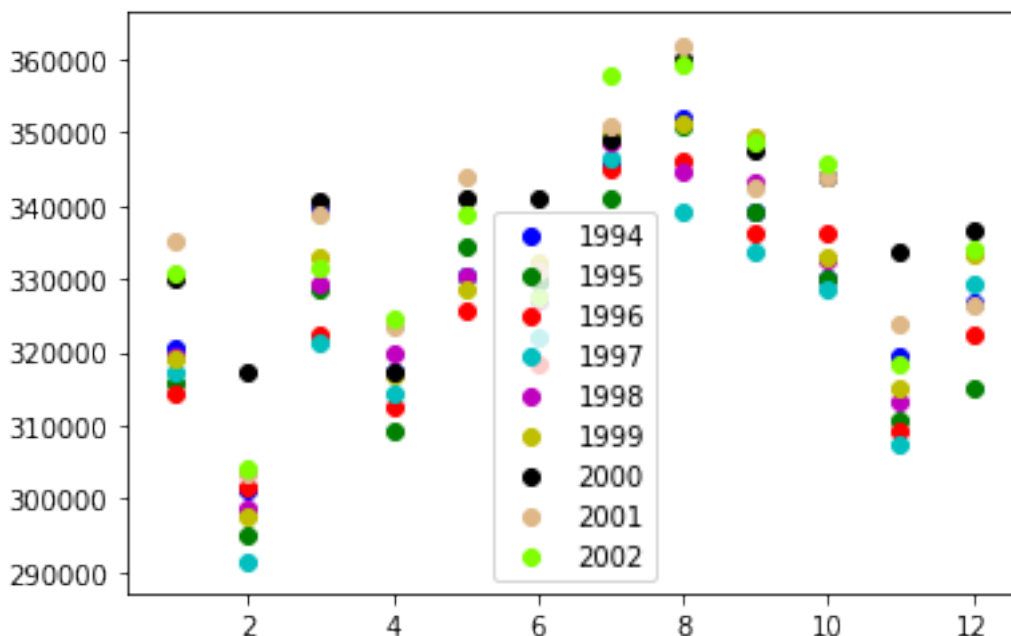
# I els anys (per a la llegenda) amb el seu valor. Podem generar la llista a mà
↳ o obtenirla
# com els valors únics de la columna years
years = data['year'].unique()

# Finalment definim una llista amb 10 colors, un per a cada any
colours = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'burlywood', 'chartreuse']

# Un cop tenim els eixos, colors i dades definides, podem generar un scatter
↳ per cada any
# cada punt a la gràfica (x, y) vindrà definit pels elements de les llistes
↳ months i year_data
# label definirà quina etiqueta se li donarà al scatter a la llegenda, i
↳ correspondrà amb l'any
for year_data, y, c in zip(parsed_years, years, colours):
    plt.scatter(months, year_data, color=c, label=y)

# I finalment incloem també la llegenda (loc=0 la posiciona "al millor lloc"
↳ dins
# del limit del gràfic). I mostrem la figura.
plt.legend(loc=0)
plt.show()

```



### 1.3.9 Exercici 9

Carregueu les dades de l'arxiu `company_sales.csv`, que podeu trobar a la carpeta `data`, en forma de `dataframe`. Aquest conjunt de dades recull informació sobre les vendes mensuals d'una empresa de cosmètica i productes d'higiene.

Part A:

Mostreu un gràfic de línies de punts de les vendes totals (*total\_profit*) per cada mes (els mesos estan numerats d'1 a 12).

*Consideracions:*

- La línia ha de ser amb punts (*dotted line*) de color vermell i de grandària 3.
- S'ha de mostrar la llegenda a la part inferior dreta.

Part B:

Mostreu un gràfic circular (*pie chart*) amb el número d'unitats venudes aquest any per producte en percentatge.

*Consideracions:*

- Heu d'incloure el percentatge (%) de cada producte dins del seu respectiu espai del gràfic.
- S'ha de mostrar la llegenda a la part dreta.

```
[24]: # Carreguem les llibreries necessàries
import pandas as pd
import matplotlib.pyplot as plt
```

```

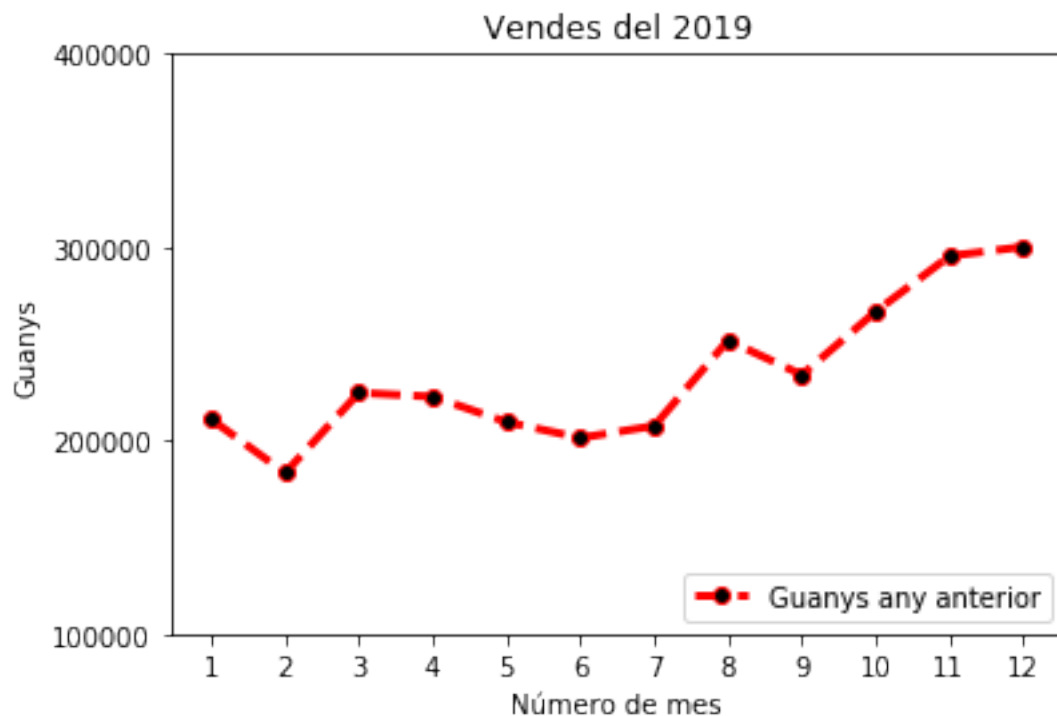
# Importem el fitxer a un dataframe de pandas
df = pd.read_csv("data/company_sales.csv")

# Seleccióem les dues columnes que ens interessin
profitList = df ['total_profit']
monthList = df ['month_number']

# Grafiquem els guanys (y) en funció del mes (x), amb les característiques
↳ demanades a l'enunciat
plt.plot(monthList, profitList, label = 'Guanys any anterior',
         color='r', marker='o', markerfacecolor='k',
         linestyle='--', linewidth=3)

# Especifiquem la informació dels eixos
plt.xlabel('Número de mes')
plt.ylabel('Guanys')
plt.legend(loc='lower right')
plt.title('Vendes del 2019')
plt.xticks(monthList)
plt.yticks([100000, 200000, 300000, 400000])
plt.show()

```



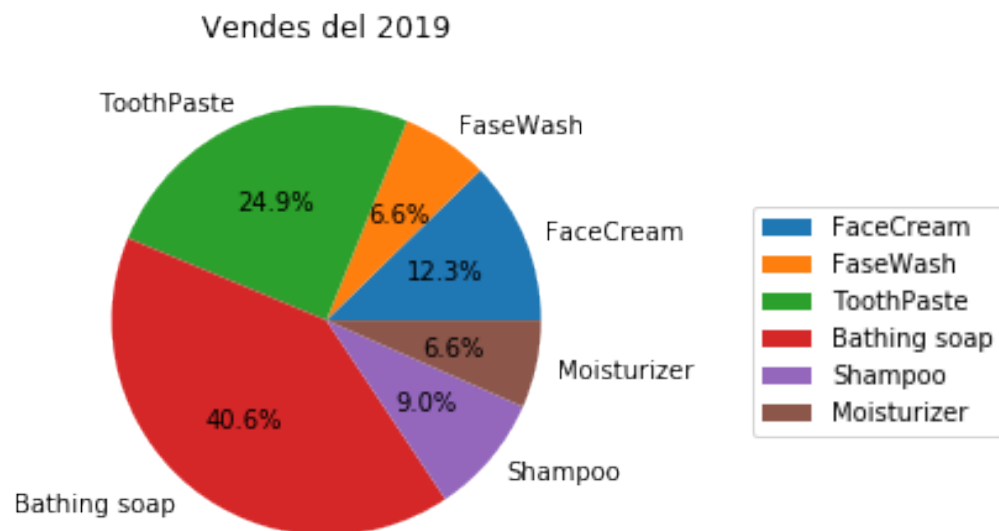
```
[25]: # Carreguem les llibreries necessàries
import pandas as pd
import matplotlib.pyplot as plt

# Importem el fitxer a un dataframe de pandas
df = pd.read_csv("data/company_sales.csv")

# Creem un vector amb la suma de cada un dels productes
salesData = [df['facecream'].sum(), df['facewash'].sum(), df['toothpaste'].
    ↳sum(),
    df['bathingsoap'].sum(), df['shampoo'].sum(), df['moisturizer'].
    ↳sum()]

# Creem la informació per la llegenda
labels = ['FaceCream', 'FaseWash', 'ToothPaste', 'Bathing soap', 'Shampoo',
    ↳'Moisturizer']

# Especifiquem la informació dels eixos
plt.axis("equal")
plt.pie(salesData, labels=labels, autopct='%1.1f%%') # autopct mostra el
    ↳percentatge (%)
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
plt.title('Vendes del 2019')
plt.show()
```



### 1.3.10 Exercici 10

Carregueu les dades de l'arxiu `got.csv` a un *dataframe*. Aquest conjunt de dades recull informació de la [Guerra dels Cinc Reis](#) de les novel·les de [Cançó de gel i de foc](#) de George R.R Martin.

Mostreu el número de files del *dataframe* i les etiquetes dels eixos.

```
[26]: # Carreguem les llibreries necessàries
import pandas as pd

# Llegim l'arxiu csv amb el metode read_csv
data = pd.read_csv('data/got.csv')

# Mostrem el numero de files
print("Número de files:", len(data))

# Mostrem les etiquetes dels eixos
print("Etiquetes:", data.axes)
```

Número de files: 38

Etiquetes: [RangeIndex(start=0, stop=38, step=1), Index(['name', 'year', 'battle\_number', 'attacker\_king', 'defender\_king', 'attacker\_1', 'attacker\_2', 'attacker\_3', 'attacker\_4', 'defender\_1', 'defender\_2', 'defender\_3', 'defender\_4', 'attacker\_outcome', 'battle\_type', 'major\_death', 'major\_capture', 'attacker\_size', 'defender\_size', 'attacker\_commander', 'defender\_commander', 'summer', 'location', 'region', 'note'], dtype='object')]

### 1.3.11 Exercici 11

Agrupeu les dades carregades a l'exercici 10 pel principal bàndol atacant (*attacker\_1*). Per cada posició, mostreu el número de batalles i el resultat de la batalla (el resultat es troba al camp *attacker\_outcome*).

```
[27]: # Tal i com estava definit l'enunciat es podia interpretar de dues formes:
# 1 - que es mostrés el número de batalles guanyades i perdudes per cada bàndol
# 2 - que es mostrés el número de batalla (identificador) i el resultat
    ↳ d'aquesta per cada
#     bàndol.
# Resoldrem els dos casos.

# Carreguem les llibreries necessàries
import pandas as pd
import numpy as np

# En qualsevol dels casos hem de carregar les dades i agrupar-les per el bàndol
    ↳ atacant
```

```

data = pd.read_csv('data/got.csv')
grouped = data.groupby('attacker_1')

# Cas 1. Cada grup conté com a etiqueta el bàndol (primera dada que ens
↳ demanen) i com
# a dades un dataframe. Podem accedir a cadascuna de les columnes com si d'un
↳ diccionari
# es tractés, i extreure els seus valors.
print("CAS 1:\n")
# Per a cada agrupació
for name, g in grouped:
    # Mostrem el bàndol
    print(name)
    # Mostrem el numero de batalla i el resultat per a cada batalla
    for battle, outcome in zip(g['battle_number'].values, g['attacker_outcome']
↳ values):
        print("battle: %s, outcome: %s" % (battle, outcome))

# Cas 2. L'estructura és la mateixa que en el cas anterior, però ara hem de
↳ comptar quantes
# batalles s'han guanyat o perdut.
print("\n-----\n")
print("CAS 2: \n")
for name, g in grouped:
    # Comencem comptant el numero de batalles extraient l'informació del
↳ dataframe
    outcome = g['attacker_outcome'].values
    s = len(outcome)

    # Després comptem quantes han estat guanyades i perdudes
    # count_nonzero avalua una expressió booleana i retorna el numero de
↳ coincidències
    # que no son zero
    w = np.count_nonzero(outcome == 'win')
    l = np.count_nonzero(outcome == 'loss')

    # Podem fer el mateix utilitzant count si transformem outcome a una llista
    outcome = list(outcome)
    w = outcome.count('win')
    l = outcome.count('loss')

    # Finalment mostrem el resultat
    print(name)
    print("total: %s, won: %s, lost: %s" % (s, w, l))

```



CAS 1:

Baratheon

battle: 16, outcome: win  
battle: 20, outcome: loss  
battle: 31, outcome: win  
battle: 34, outcome: win  
battle: 35, outcome: win  
battle: 38, outcome: nan

Bolton

battle: 14, outcome: win  
battle: 29, outcome: win

Bracken

battle: 37, outcome: win

Brave Companions

battle: 30, outcome: win

Brotherhood without Banners

battle: 23, outcome: win

Darry

battle: 21, outcome: win

Free folk

battle: 28, outcome: loss

Frey

battle: 26, outcome: win  
battle: 27, outcome: win

Greyjoy

battle: 8, outcome: win  
battle: 9, outcome: win  
battle: 10, outcome: win  
battle: 12, outcome: win  
battle: 13, outcome: win  
battle: 32, outcome: win  
battle: 33, outcome: win

Lannister

battle: 1, outcome: win  
battle: 2, outcome: win  
battle: 3, outcome: win  
battle: 7, outcome: win  
battle: 17, outcome: loss  
battle: 24, outcome: win  
battle: 25, outcome: win  
battle: 36, outcome: win

Stark

battle: 4, outcome: loss  
battle: 5, outcome: win  
battle: 6, outcome: win  
battle: 11, outcome: win  
battle: 15, outcome: win

```
battle: 18, outcome: win
battle: 19, outcome: win
battle: 22, outcome: loss
```

-----

CAS 2:

```
Baratheon
total: 6, won: 4, lost: 1
Bolton
total: 2, won: 2, lost: 0
Bracken
total: 1, won: 1, lost: 0
Brave Companions
total: 1, won: 1, lost: 0
Brotherhood without Banners
total: 1, won: 1, lost: 0
Darry
total: 1, won: 1, lost: 0
Free folk
total: 1, won: 0, lost: 1
Frey
total: 2, won: 2, lost: 0
Greyjoy
total: 7, won: 7, lost: 0
Lannister
total: 8, won: 7, lost: 1
Stark
total: 8, won: 6, lost: 2
```

### 1.3.12 Exercici 12

Mostreu les dades de las batalles on el número de participants supera els 15000 “homes” (comptant els dos bàndols: *attacker\_size* i *defender\_size*), el resultat hagi estat desfavorable per l'atacant, i la batalla hagi estat a l'hivern ( *summer* = 0).

```
[28]: # Carreguem les llibreries necessàries
import pandas as pd

# Carreguem les dades
data = pd.read_csv('data/got.csv')

# Definim les condicions
# La suma de les tropes atacants i defensores ha de ser superior a 15000
size_cond = (data['attacker_size'] + data['defender_size'] > 15000)
# El resultat ha de ser desfavorable per l'atacant
```

```

outcome_cond = data['attacker_outcome'] == 'loss'
# I la batalla ha d'haver succeït a l'hivern
season_cond = data['summer'] == 0

# Accedim a les dades combinant totes les condicions i les mostrem per pantalla
print(data[size_cond & outcome_cond & season_cond])

# Podem veure que es tracta d'una única batalla (battle_number = 28),
↳ corresponent a
# Battle of Castle Black

```

```

              name year battle_number attacker_king \
27 Battle of Castle Black 300          28 Stannis Baratheon

defender_king attacker_1 attacker_2 attacker_3 attacker_4 defender_1 \
27 Mance Rayder Free folk   Thenns   Giants      NaN Night's Watch

... major_death major_capture attacker_size defender_size \
27 ...          1.0           1.0       100000.0      1240.0

              attacker_commander \
27 Mance Rayder, Tormund Giantsbane, Harma Dogshe...

              defender_commander summer location \
27 Stannis Baratheon, Jon Snow, Donal Noye, Cotte... 0.0 Castle Black

region note
27 Beyond the Wall NaN

[1 rows x 25 columns]

```

### 1.3.13 Exercici 13

Compteu quants llocs apareixen més d'un cop al *dataframe* (camp *location*), utilitzant les funcions de la llibreria *pandas*. Existeix algun lloc on hi hagi hagut més d'una batalla? Comproveu quins bàndols hi estaven implicats.

```

[29]: # Carreguem les llibreries necessàries
import pandas as pd

# Carreguem les dades
data = pd.read_csv('data/got.csv')

# Hi ha diferents formes de resoldre el problema. Una d'elles seria utilitzant
↳ el mètode
# value_counts que donada una columna ens retorna el número d'ocurrències dels
↳ valors d'aquesta

```

```

# Podem cercar dins del dataframe de forma similar a l'exercici anterior
↳ utilitzant > 1
locations = data['location'].value_counts()
rep_loc = locations[locations > 1]
print("Número de llocs que apareixen més d'un cop al dataframe:", len(rep_loc))

# De forma alternativa podem agrupar les dades per localització i computar la
↳ mida dels grups
# (quants cops apareix cadascun)
grouped = data.groupby('location')
locations = grouped.size()

# Podem accedir al dataframe utilitzant el mètode loc i passant una llista
↳ booleana de posicions
# corresponents a si el contingut de la posició es més gran que 1. Per això
↳ podem utilitzar
# una lambda function
rep_loc2 = locations.loc[lambda x: x>1]
print("Número de llocs que apareixen més d'un cop al dataframe (v2):",
↳ len(rep_loc2))

# Per mostrar els bàndols principals implicats al llocs on hi ha hagut més
↳ d'una batalla
# podem accedir a les files del dataframe que tenen com a localització els
↳ llocs repetits
# que acabem d'obtenir. Podem utilitzar qualsevol identificatiu dels bàndols:
# attacker_1 vs defender_1, attacker_kings vs defender_king, etc

# Recorrem la llista de localitzacions repetides
for loc, c in rep_loc.iteritems():
    # Mostrem la localització
    print("\n%s:" % loc)
    # Definim el filtre per aquesta localització i filtrem les batalles que han
↳ tingut lloc
    # aquí
    loc_cond = data["location"] == loc
    battles = data[loc_cond]
    # Finalment mostrem els bàndols
    for a, d in zip(battles['attacker_1'], battles['defender_1']):
        print("%s vs %s" %(a,d))

```

Número de llocs que apareixen més d'un cop al dataframe: 8

Número de llocs que apareixen més d'un cop al dataframe (v2): 8

Winterfell:

Greyjoy vs Stark

Bolton vs Stark

Baratheon vs Bolton

Riverrun:

Lannister vs Tully

Stark vs Lannister

Lannister vs Tully

Moat Cailin:

Greyjoy vs Stark

Bolton vs Greyjoy

Deepwood Motte:

Greyjoy vs Stark

Baratheon vs Greyjoy

Storm's End:

Baratheon vs Baratheon

Baratheon vs Baratheon

Torrhen's Square:

Stark vs Greyjoy

Greyjoy vs Stark

Darry:

Lannister vs Darry

Darry vs Lannister

Harrenhal:

Stark vs Lannister

Lannister vs Brave Companions

### 1.3.14 Exercici 14

Carregueu les dades de l'arxiu `netflixtitles.csv`, que podeu trobar a la carpeta `data`, en un *dataframe*. Aquest conjunt de dades recull informació sobre pel·lícules i sèries de TV de Netflix (<https://www.kaggle.com/shivamb/netflix-shows>) fins el 2019.

Mostreu les pel·lícules estrenades el 2019 que tenen una duració superior a 100 minuts.

**Nota:** Al Notebook de teoria hem vist operacions bàsiques de filtrat. Per resoldre aquest exercici us caldrà investigar com aplicar condicions més complexes.

**Nota 2:** Els valors de la columna de duració (*duration*) són strings, ja que combinen números i lletres. Per tant, s'han de transformar a int un cop s'hagi construït el subconjunt de dades de pel·lícules. Us pot ser d'utilitat la funció `astype` de pandas.

```
[30]: # Carreguem les llibreries necessàries
import numpy as np
```

```

import pandas as pd

# Importem el fitxer a un dataframe de pandas
data = pd.read_csv("data/netflixtitles.csv")

# Filtrem el dataset pel tipus "Movie"
data_movies=data[data['type']=="Movie"]

# Podem mostrar els 10 primers valors únics de la columna duration de la
↳següent manera.
print(data_movies['duration'][0:10])

# Veiem que els minuts els agafa com a string, els hem de convertir a número.
data_movies['duration']= data_movies['duration'].astype(int)

# Extraiem els valors que ens interessin dels valors mostrats
# L'operador & ens permet aplicar un operador lògic AND
data_ex3 = data_movies[(data_movies["release_year"] == 2019) &
↳(data_movies["duration"] > 100)]

# Accedim a la columna "title" del conjunt filtrat i mostrem els seus valors
data_ex3['title']

```

```

0      90
1      94
4      99
6     110
7      60
9      90
10     78
11     95
12     58
13     62

```

Name: duration, dtype: object

```

[30]: 35      Article 15
      57      The World We Make
      102     In the Shadow of the Moon
      109      Malaal
      119     Oh! Baby (Malayalam)
      ...
      5347    Ek Ladki Ko Dekha Toh Aisa Laga
      5359      Music Teacher
      5365      The Ruthless
      5385    HOMECOMING: A film by Beyoncé
      5469      706

```

Name: title, Length: 134, dtype: object

[ ]: